



# Relatório Projeto II - identificação de prefixos e indexação de dicionários

Maria Eduarda Teixeira Costa e Sofia Gazolla da Costa Silva

Departamento de Informática e Estatística - Universidade Federal de Santa Catarina

Estrutura de Dados

Professores: Alexandre Goncalves Silva e Aldo Von Wangenheim

Novembro 2025

## 1 Introdução

O Projeto II consistiu na implementação e utilização da estrutura hierárquica *trie* para a indexação e recuperação eficiente de palavras em grandes arquivos de dicionários.

A implementação deveria resolver dois problemas principais. O primeiro era verificar se entradas fornecidas pelo usuário eram prefixos de palavras presentes no dicionário e, caso fossem, indicar não apenas que o prefixo é válido, mas também quantas palavras o possuem. Já o segundo problema tratava da situação em que a entrada do usuário correspondia a uma palavra completa do dicionário. Nesse caso, o programa deveria identificar sua posição no arquivo e o comprimento da linha em que ela aparece.

## 2 Desenvolvimento - Problema 1

### 2.1 Lógica da Função

Para a resolução da primeira atividade, que visava identificar se uma determinada entrada é um prefixo válido de palavras no dicionário, foram implementadas as funções

`buscarNo()` e `contarPalavras()`. Para isso, utilizamos a estrutura de dados **trie** (árvore de prefixos).

O método consiste em percorrer a trie a partir da raiz, seguindo o caminho determinado por cada caractere do prefixo fornecido. A função `buscarNo()` navega pela estrutura convertendo cada caractere em um índice numérico (subtraindo 'a' do caractere). Se em algum ponto do caminho não existir um nó filho correspondente ao caractere, isso significa que o prefixo não existe na trie, e a função retorna `nullptr`.

Caso o prefixo seja encontrado, a função `contarPalavras()` é chamada recursivamente a partir do nó final do prefixo. Esta função percorre toda a subárvore contando quantas palavras completas existem (identificadas quando `comprimento > 0`). O contador é incrementado para cada palavra encontrada e somado recursivamente para todos os filhos do nó atual, garantindo que todas as palavras com aquele prefixo sejam contabilizadas.

## 2.2 Implementação

```
NoTrie* buscarNo(NoTrie* raiz, string prefixo) {
    NoTrie* atual = raiz;

    for (char c : prefixo) {
        int index = c - 'a';

        if (atual->filhos[index] == nullptr) {
            return nullptr;
        }

        atual = atual->filhos[index];
    }

    return atual;
}

int contarPalavras(NoTrie* no) {
    if (no == nullptr) {
        return 0;
    }

    int contador = 0;

    if (no->comprimento > 0) {
        contador = 1;
    }

    for (int i = 0; i < tamanho_alfabeto; i++) {
        contador += contarPalavras(no->filhos[i]);
    }

    return contador;
}
```

Figure 1: Funções `buscarNo()` e `contarPalavras()`

## 2.3 Imagem

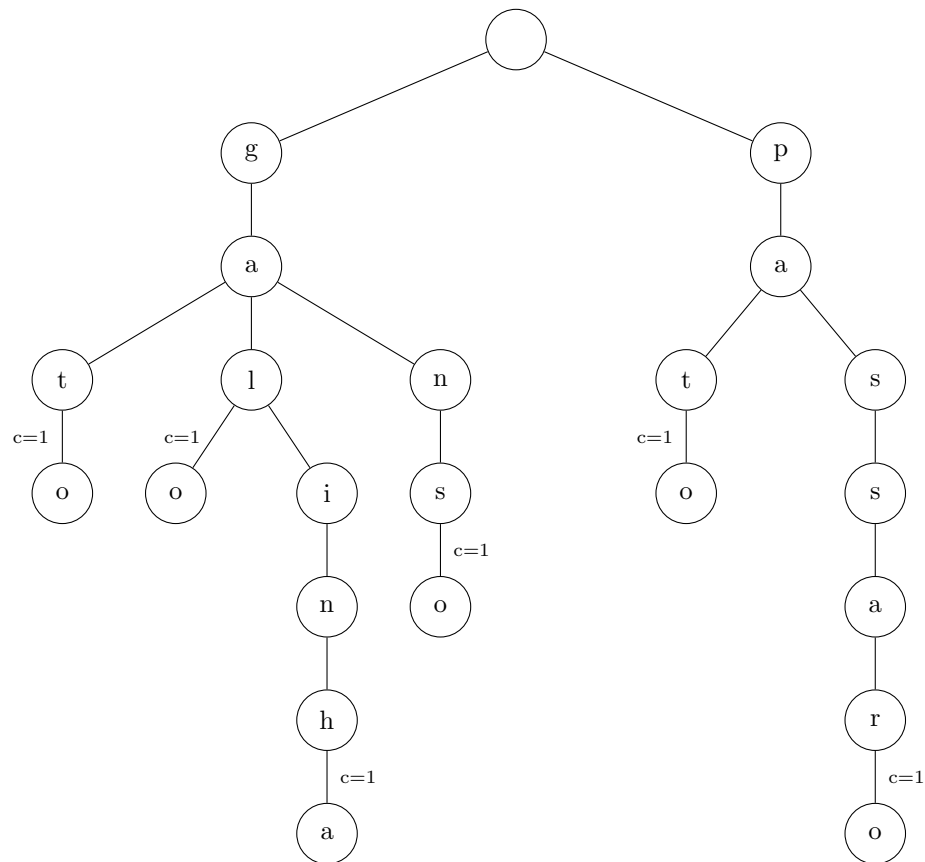


Figure 2: Exemplo de Trie com palavras: gato, galo, galinha, ganso, pato, passaro. Pode-se perceber que para o prefixo "ga", por exemplo, haveria quatro palavras, pois há quatro filhos saindo de  $g \rightarrow a$ .

## 3 Desenvolvimento - Problema 2

### 3.1 Lógica da Função

O método `inserir()` implementa a construção da trie e a indexação das palavras do dicionário. Esta função recebe como parâmetros a palavra a ser inserida, sua posição no arquivo e o comprimento da linha.

A inserção ocorre percorrendo cada caractere da palavra e criando nós conforme necessário. Para cada caractere, o algoritmo converte a letra em um índice numérico (0-25) e verifica se já existe um nó filho naquela posição. Se não existir, um novo nó é criado utilizando a função `criarNo()`, que aloca memória e inicializa todos os ponteiros como `nullptr` e os valores de posição e comprimento como zero.

O algoritmo avança pela estrutura até chegar ao último caractere da palavra. Neste ponto final, são armazenadas as informações de indexação: a **posição** indica onde a palavra começa no arquivo (incluindo o colchete de abertura), e o **comprimento** guarda o tamanho total da linha (excluindo o caractere de quebra de linha). A presença de um valor de **comprimento** > 0 serve como marcador de que aquele nó representa o final de uma palavra completa no dicionário.

Durante a leitura do arquivo, a função `tellg()` é utilizada para rastrear a posição atual no arquivo, permitindo que cada palavra seja corretamente indexada com sua localização física no disco.

### 3.2 Implementação

```
NoTrie* criarNo() {
    NoTrie* no = new NoTrie;

    no->posicao = 0;
    no->comprimento = 0;

    for (int i = 0; i < tamanho_alfabeto; i++) {
        no->filhos[i] = nullptr;
    }

    return no;
}

void inserir(NoTrie* raiz, string palavra, unsigned long pos, unsigned long comp) {
    NoTrie* atual = raiz;

    for (char c : palavra) {
        int index = c - 'a';

        if (atual->filhos[index] == nullptr) {
            atual->filhos[index] = criarNo();
        }

        atual = atual->filhos[index];
    }

    atual->posicao = pos;
    atual->comprimento = comp;
}
```

Figure 3: Funções `criarNo()` e `inserir()`

### 3.3 Imagem

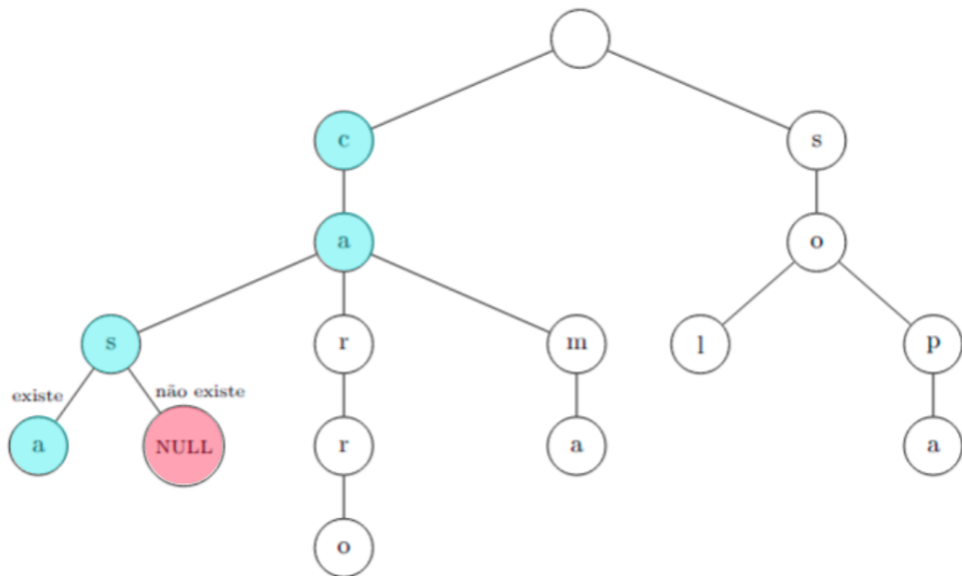


Figure 4: Busca pela palavra “caso”. Após percorrer  $c \rightarrow a \rightarrow s$ , não existe filho “o”, causando falha na busca.

## 4 Dificuldades

O primeiro desafio significativo do projeto foi o correto gerenciamento da memória dinâmica na estrutura da trie. Como cada nó possui um vetor de 26 ponteiros para possíveis filhos, era fundamental implementar uma função recursiva de liberação de memória (`liberarMemoria()`) que percorresse toda a árvore de forma pós-ordem, liberando primeiro os filhos e depois o nó atual. Isso evitou vazamentos de memória que poderiam ocorrer ao encerrar o programa.

Outro desafio foi o correto tratamento da extração das palavras do arquivo de dicionário. Inicialmente, houve dificuldade em identificar precisamente onde cada palavra começava e terminava, considerando o formato `[palavra] definição`. A solução encontrada foi utilizar a função `find(']')` para localizar o colchete de fechamento e então extrair a substring desde a posição 1 (logo após o `'['`) até a posição do `']'`. Além disso, foi necessário verificar se o colchete foi encontrado (`posFechamento != string::npos`) para evitar processar linhas malformadas.

## References

- [1] INSTITUTO DE MATEMÁTICA E ESTATÍSTICA - USP. *Tries*. Disponível em: <https://www.ime.usp.br/~pf/estruturas-de-dados/aulas/tries.html>. Acesso em: 18 nov. 2025.
- [2] GEEKSFORGEEKS. *Trie Insert and Search*. Disponível em: <https://www.geeksforgeeks.org/trie-insert-and-search/>. Acesso em: 18 nov. 2025.
- [3] PATEL, Bhargav. *Implementing a Trie data structure in Python in less than 100 lines of code*. Medium, 2018. Disponível em: <https://medium.com/data-science/implementing-a-trie-data-structure-in-python-in-less-than-100-lines-of-code/>. Acesso em: 18 nov. 2025.
- [4] CPLUSPLUS.COM. *string::find — cplusplus.com reference*. Disponível em: <https://cplusplus.com/ref/string/string/find/>. Acesso em: 18 nov. 2025.
- [5] CPLUSPLUS.COM. *string::substr — cplusplus.com reference*. Disponível em: <https://cplusplus.com/reference/string/string/substr/>. Acesso em: 18 nov. 2025.
- [6] CPLUSPLUS.COM. *basic\_istream::tellg — cplusplus.com reference*. Disponível em: <https://cplusplus.com/reference/istream/istream/tellg/>. Acesso em: 18 nov. 2025.